



中山大學
SUN YAT-SEN UNIVERSITY

数据抽象和类 I

中山大学计算机学院



主讲人：郑贵锋

zhenggf@mail.sysu.edu.cn



中山大学MOOC课程组

目录

CONTENTS

01

预备知识

02

访问控制（初步）

03

构造/析构 函数

04

This 指针

05

应用：Date 类示例



预备：C++新类型 - bool

C语言：

- 没有 **bool** 类型，c 程序员通常用

```
#define TRUE=1
#define FALSE=0
```
- C99 定义了 `_Bool` 类型，并通过 `stdbool.h` 实现与 C++ 兼容

C++语言：

- 定义了三个关键字：bool, true, false
- 当显式（例 `(bool)7`）或隐式（例 `bool b=7;`）转为 bool 类型时
 - 0 值转为 false
 - 非 0 值转为 true
- 使用 `cout` 输出

```
cout << true 输出是 1
cout << false 输出是 0
```



预备：C++形参 - void

C语言：

- func(); 没有声明形参表示函数的形参不确定
- 没有参数，则**必须显式声明** func(void);

C++语言：

- func(); 等价于 func(void);
- C++则使用 "..." 标点符号表示可变参数

// 函数声明如下

```
int printx(const char* fmt, ...);
```

// 能以一个或多个实参调用：

```
printx("hello world");
```

```
printx("a=%d b=%d", a, b);
```



预备： C++新特性 - 函数重载

问题：

在开发中，需要的函数功能类似，但参数数量或类型不同。
例如，交换整数和交换浮点数两个函数

C语言解决方法：

- 必须申明两个函数，否则会标识符重定义。例如：

```
void SwapInt(int * x,int * y);  
void SwapDouble(double * x,double * y);
```

C++语言解决方法：

- 声明同样名称的一组函数，但参数类型不同：

```
void Swap(int& x,int& y);  
void Swap(double& x,double& y);
```



预备：C++新特性 - 函数重载 与 auto

优点：

- 程序更易于理解
- 两个实现唯一差别是 temp 类型不同
- 使用 auto 类型，则实现代码完全一样
- 让编译去**推导**决定哪个函数

函数签名：

- 编译器将函数名、参数数量、参数类型编译为唯一的内部函数名。
- 例如：Swap 函数。Swap_int_int 和 Swap_float_float 是不同函数签名，所以不是一个函数
- 函数签名是复杂的，后面课程详解

```
#include <iostream>
using namespace std;

void Swap(int& x,int& y);
void Swap(double& x,double& y);

void Swap(int& x,int& y) {
    auto temp = x; x = y; y = temp;
}

void Swap(double& x,double& y) {
    auto temp = x; x = y; y = temp;
}

int main() {
    int i = 2,j = 3;
    Swap(i,j);
    cout << i << "," << j << endl;
    double x = 2.2, y = 3.3;
    Swap(x,y);
    cout << x << "," << y << endl;
}
```



预备： C++新特性 - 默认实参

C语言：

- **不支持**函数默认参数和值

C++语言：

- 默认参数并非编程方面的重大突破，而只是提供了一种便捷的方式。
- 默认参数只能定义在参数**右边**
- 一个声明和实现，可匹配多个函数
- 例如：
 - 右例申明可匹配的函数，包括：
 - func()
 - func(int)
 - func(int,float)
 - func(int,float,char)

```
#include<iostream>
using namespace std;
//带默认参数的函数申明
void func(int n=1, float b=1.2, char c='@');
//实现
void func(int n, float b, char c){
    cout<<n<<"", "<<b<<"", "<<c<<endl;
}

int main(){
    func();                //func(1,1.2,'@')
    func(10);              //func(10,1.2,'@')
    func(20, 9.8);         //func(20,9.8,'@')
    func(30, 3.5, '#');    //func(30,3.5,'#')
    return 0;
}
```



预备：字符串类型 - string

C语言字符串：

- C 字符串是 `char *` 类型，是以 `' \0 '` 字符结束的字符数组。
`const char *s = "c plus plus" ;`
- 在C++中处理 C 字符串，请使用 `#include <cstring>`

C++语言字符串：

- `string` 是类
- 作为区别，用 `cstring` 称c字符数组
- 阅读右边程序，思考
 - 对象为什么必须构造？
 - 为什么需要那么多构造方法？
 - `s2` 和 `s3` 为什么不是赋值？
 - 移出作用域时，对象如何回收？

```
#include <iostream>
#include <string>
using namespace std;

int main(){
    string s1;    // 默认构造
    string s2 = "c plus plus"; // 用cstring构造
    string s3 = s2; // 用同类对象构造
    string s4 (5, 's'); // 用int,char作为参数构造
    cout << s2 << endl;
    cout << s4 << endl;
    return 0;
}
```



访问控制：公有（public）成员

公有成员在客户端可以任意访问。**公有数据成员**不需要通过公有函数成员访问，其优点是使用方便，缺点是可能会**破坏封装**的逻辑一致性，如客户端修改线段长度为负数。

```
/* public.hpp */
#pragma once

namespace sysu_cplus {
    class Line {
    public:
        double length;
        void setLength( double len );
        double getLength( void );
    };
}

// 成员函数定义
double sysu_cplus::Line::getLength(void) {
    return length ;
}
void sysu_cplus::Line::setLength( double len ) {
    length = len;
}
```



访问控制：公有（public）成员

公有成员在客户端可以任意访问。**公有数据成员**不需要通过公有函数成员访问，其优点是使用方便，缺点是可能会**破坏封装**的逻辑一致性，如客户端修改线段长度为负数。

```
#include<iostream>
#include "public.hpp"
using namespace std;
using namespace sysu_cplus;

// 程序的主函数
int main( ){
    Line line;
    // 设置长度
    line.setLength(6.0);
    cout << "Length of line : " << line.getLength() <<endl;
    // 不使用成员函数设置长度
    line.length = 10.0;
    // OK: 因为 length 是公有的
    cout << "Length of line : " << line.length <<endl;
    // Error: 破坏包装的后果
    line.length = -10.0;
    cout << "Length of line : " << line.getLength() <<endl;
    return 0;
}
```

输出结果：

Length of line : 6

Length of line : 10



访问控制：私有（private）成员

私有成员变量或函数在类的外部是不可访问的，甚至是不可查看的。只有**类**和**友元函数**可以访问私有成员。

默认情况下，**class**的所有成员都是私有的。例如在右面的类中，**width** 是一个私有成员，这意味着，如果您没有使用任何访问修饰符，类的成员将被假定为私有成员。

```
class Box {  
    double width;  
    public:  
        double length;  
        void setWidth( double wid );  
        double getWidth( void );  
};
```

注：友元函数后边课程有讲解，在理论选择题中别出错哦！



访问控制：私有（private）成员

实际操作中，我们一般会在私有区域定义数据，在公有区域定义相关的函数，以便在类的外部也可以调用这些函数，如下所示：

```
/* private.hpp */
namespace sysu_cplus {
    class Box {
    public:
        double length;
        void setWidth( double wid );
        double getWidth(void) const;
        double getArea();
    private:
        double width;
    };
}
// 在函数定义中，我们如何区别 namespace名 和 类名？
double sysu_cplus::Box::getWidth(void) const{
    return width ;
}
void sysu_cplus::Box::setWidth(double w) {
    width = w;
}
// 请给出面积计算的实现
// 请课后改写该类的声明，使得包装逻辑正确
```



访问控制：私有（private）成员

作为**标准封装**手段，一般会在私有区域定义数据，在公有区域定义相关的函数。以便在类的外部**仅可以调用成员函数修改对象状态**，保持对象内部状态一致。如下所示：

```
...
int main() {
    Box box;
    // 不使用成员函数设置长度
    box.length = 10.0;
    // OK: 因为 length 是公有的
    cout << "Length of box : " << box.length << endl;
    // 不使用成员函数设置宽度
    // box.width = 10.0;
    // Error: 因为 width 是私有的
    // 使用成员函数设置宽度
    box.setWidth(10.0);
    cout << "Width of box : " << box.getWidth() << endl;
    // 请计算并输出面积，保留两位小数
    // 思考，将长度赋值负数，负面后果有哪些？
    return 0;
}
```

输出结果：

Length of box : 10

Width of box : 10



构造函数 - 无参构造函数

类的构造函数是类的一种特殊的成员函数，每次创建类的新对象时执行它完成初始化等逻辑。

构造函数的名称与类的名称是完全相同的，并且**不会返回任何类型**，也不会返回 void。

如果用户没有自定义构造函数，则编译会自动生成一个**默认构造函数**。

```
/* constractor.hpp */
#pragma once
#include<iostream>

namespace sysu_cplus {
    class Line {
    public:
        void setLength( double len );
        double getLength( void );
        Line(); // 这是构造函数
    private:
        double length;
    };
}
// 成员函数定义，包括构造函数
sysu_cplus::Line::Line(void) {
    std::cout << "Object is being created" << std::endl;
}
void sysu_cplus::Line::setLength( double len ) {
    length = len;
}
double sysu_cplus::Line::getLength( void ) {
    return length;
}
```



构造函数 - 无参构造函数

类的构造函数是类的一种特殊的成员函数，每次创建类的新对象时执行它完成初始化等逻辑。

构造函数的名称与类的名称是完全相同的，并且**不会返回任何类型**，也不会返回 void。

如果用户没有自定义构造函数，则编译会自动生成一个**默认构造函数**。

```
#include <iostream>
#include "constructor.hpp"
using namespace std;
using namespace sysu_cplus;

// 程序的主函数
int main() {
    Line line;
    // 设置长度
    line.setLength(6.0);
    cout << "Length of line : " << line.getLength() << endl;
    return 0;
}
```

输出结果：

Object is being created
Length of line : 6



构造函数 - 带参数的构造函数

构造函数也可以带有参数。这样在创建对象时就可使用参数构造对象。

注意：用户一旦定义了构造函数，编译器就**不再自动添加**默认构造函数。这是调用无参构造会出错

如下面的例子所示：

```
/* constructor_para.hpp */
...
namespace sysu_cplus {
    class Line {
    public:
        Line(double len); //构造函数
        void setLength( double len );
        double getLength( void );
    private:
        double length;
    };
}
// 成员函数定义，包括构造函数
sysu_cplus::Line::Line(double len) {
    length = len;
    std::cout << "Object is being created, length = " << len << std::endl;
}
void sysu_cplus::Line::setLength( double len ) {
    length = len;
}
double sysu_cplus::Line::getLength( void ) {
    return length;
}
```



构造函数 - 带参数的构造函数

构造函数也可以带有参数。这样在创建对象时就可使用参数构造对象。

注意：用户一旦定义了构造函数，编译器就**不再自动添加**默认构造函数。这是调用无参构造会出错

如下面的例子所示：

```
#include <iostream>
#include "constructor_para.hpp"
using namespace std;
using namespace sysu_cplus;

// 程序的主函数
int main( ) {
    Line line(10.0);
    Line line1; //Error, 没有默认构造
    // 获取默认设置的长度
    cout << "Length of line : " << line.getLength() << endl;
    // 再次设置长度
    line.setLength(6.0);
    cout << "Length of line : " << line.getLength() << endl;
    return 0;
}
```

输出结果：

Object is being created, length = 10

Length of line : 10

Length of line : 6



构造函数 - 默认实参构造函数

构造函数也能使用默认实参。

这样可以减少构造函数重载的数量。

如下面的例子所示，一个实现可用于多种构造，如：

Line line, line1=10;

请自己完成客户端测试代码

```
/* constructor_default_para.hpp */
...
namespace sysu_cplus {
    class Line {
    public:
        Line(double len = 0); //构造函数
        void setLength( double len );
        double getLength( void );
    private:
        double length;
    };
}
// 成员函数定义，包括构造函数
sysu_cplus::Line::Line(double len) {
    length = len;
    std::cout << "Object is being created, length = " << len << std::endl;
}
void sysu_cplus::Line::setLength( double len ) {
    length = len;
}
double sysu_cplus::Line::getLength( void ) {
    return length;
}
```



析构函数

类的析构函数是类的一种特殊的成员函数，它会在对象被释放前执行。

析构函数的名称与类的名称是完全相同的，只是在前面加了个波浪号（~）作为前缀，它不会返回任何值，也不能带有任何参数。

析构函数有助于在跳出程序（比如关闭文件、释放内存等）前释放资源。下面的实例有助于更好地理解析构函数的概念：

注意：析构函数也不能直接调用

```
/* constructor.hpp */
...
namespace sysu_cplus {
    class Line {
    public:
        void setLength( double len );
        double getLength( void );
        Line(); //构造函数
        ~Line(); //析构函数
    private:
        double length;
    };
}
// 成员函数定义，包括构造函数
sysu_cplus::Line::Line(void) {
    std::cout << "Object is being created" << std::endl;
}
sysu_cplus::Line::~~Line(void) {
    std::cout << "Object is being deleted" << std::endl;
}
...
```



析构函数

类的析构函数是类的一种特殊的成员函数，它会在对象被释放前执行。

析构函数的名称与类的名称是完全相同的，只是在前面加了个波浪号（~）作为前缀，它不会返回任何值，也不能带有任何参数。

析构函数有助于在跳出程序（比如关闭文件、释放内存等）前释放资源。下面的实例有助于更好地理解析构函数的概念：

注意：析构函数也不能直接调用

```
#include <iostream>
#include "destructor.hpp"
using namespace std;
using namespace sysu_cplus;

// 程序的主函数
int main( ) {
    Line line;
    //设置长度
    line.setLength(6.0);
    cout << "Length of line : " << line.getLength() << endl;
    return 0;
}
```

输出结果：

Object is being created

Length of line : 6

Object is being deleted



This指针

在 C++ 中，每一个对象都能通过 **this** 指针来访问自己的地址。**this** 指针是所有成员函数的隐含参数。因此，在成员函数内部，它可以用来指向调用对象。
友元函数没有 **this** 指针，因为友元不是类的成员。只有**动态**成员函数才有 **this** 指针。
下面的实例有助于更好地理解 this 指针的概念：

-> 是指针取成员运算，与 c 语言一致。

static 成员不能使用 this，应使用

类名::成员

当成员函数参数与成员数据重名时，

必须用 this 访问成员数据

```
#include <iostream>
using namespace std;
class Box {
public:
    // 构造函数定义
    Box(double l=2.0, double b=2.0, double h=2.0) {
        cout << "Constructor called." << endl;
        length = l;
        breadth = b;
        height = h;
    }
    double Volume() {
        return length * breadth * height;
    }
    int compare(Box box) {
        return this->Volume() > box.Volume();
    }
private:
    double length; // Length of a box
    double breadth; // Breadth of a box
    double height; // Height of a box
};
```



This指针

在 C++ 中，每一个对象都能通过 **this** 指针来访问自己的地址。**this** 指针是所有成员函数的隐含参数。因此，在成员函数内部，它可以用来指向调用对象。友元函数没有 **this** 指针，因为友元不是类的成员。只有成员函数才有 **this** 指针。下面的实例有助于更好地理解 this 指针的概念：

```
int main(void) {  
    Box Box1(3.3, 1.2, 1.5); // Declare box1  
    Box Box2(8.5, 6.0, 2.0); // Declare box2  
    if(Box1.compare(Box2)) {  
        cout << "Box2 is smaller than Box1" <<endl;  
    } else {  
        cout << "Box2 is equal to or larger than Box1" <<endl;  
    }  
    return 0;  
}
```

输出结果：

Constructor called.

Constructor called.

Box2 is equal to or larger than Box1



构造函数例子-Date类

// DATE.h----类DATE的说明文件

class DATE

{

public:

DATE(int, int, int); //有参构造函数
DATE(); //无参构造函数

void Set(int, int, int);

int getMonth() const;

int getDay() const;

int getYear() const;

void Print() const;

void Increment();

void Decrement();

:

构造函数重载

const 成员函数

private:

int month;

int day;

int year;

};

封装成员数据



构造函数例子-Date类

```
//DATE.cpp ---- 类DATE中各成员函数的实现
//除增加了构造函数的定义外，其他成员函数与之前的DATE.cpp相同
:
DATE::DATE( int initYear, int initMonth, int initDay )
{
    year = initYear;           //在构造函数中进行初始化
    month = initMonth;
    day  = initDay;
}
DATE::DATE()
{
    year = 2000;
    month = 1;
    day  = 1;
}
:
```



构造函数例子 - Date类

```
//client.cpp ---- 类DATE中各成员函数的实现
#include "DATE.h"
#include <iostream>
using namespace std;

int main()
{
    DATE date1; //①调用无参构造函数
    DATE date2( 1976, 12, 20 ); //②自动调用另一个有参数的构造函数

    date1.print();
    date2.print();

    return 0;
}
```



构造函数例子 - 默认实参构造函数

如果设计的类没有构造函数，C++编译器会自动为该类型建立一个缺省构造函数。该构造函数没有任何形参，且函数体为空。应该养成编写构造函数的习惯。

```
class DATE
{
    public:
        DATE( int = 2000, int = 1, int = 1 );
        :
};

DATE::DATE( int initYear, int initMonth, int initDay )
{
    year = initYear;  month = initMonth;  day  = initDay;
}
```

C++11以后建议声明写形式参数

```
int main() {
    DATE  date1;
    DATE  date2( 1976 );
    DATE  date2( 1976, 12 );
    DATE  date2( 1976, 12, 20 );
}

//client.cpp
//initYear, initMonth, initDay分别为2000, 1, 1
//initYear, initMonth, initDay分别为1976, 1, 1
//initYear, initMonth, initDay分别为1976, 12, 1
//initYear, initMonth, initDay分别为1976, 12, 10
```



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



中山大学MOOC课程组